

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Bachelor of Technology | Final Year Project Report



SATYACHECK

An AI-Powered Fake News Detection and Verification System

Using Multi-Model Artificial Intelligence (OpenAI · Google Gemini · Anthropic Claude)
with Real-Time Fact Verification and Media Literacy Education

SUBMITTED BY

Student 1 — Backend Developer | Roll No. ____

Student 2 — Frontend Developer | Roll No. ____

Student 3 — AI/ML Engineer | Roll No. ____

Student 4 — Database & DevOps | Roll No. ____

SUBMITTED TO

Prof. / Dr. _____

Department of Computer Science & Engineering



TABLE OF CONTENTS

1. Abstract	3
2. Project Overview & Novelty Factor	3
3. Technology Stack	4
4. System Architecture	4
5. Backend Engineering Concepts Learned	5
5.1 REST API Design	5
5.2 OpenAPI Contract-First Development	5
5.3 Input Validation using Zod	6
5.4 Database Design (PostgreSQL + Drizzle ORM)	6
5.5 Multi-Model AI Integration	7
5.6 Middleware Architecture	7
5.7 Structured Logging	8
5.8 Environment Variables & Secret Management	8
5.9 Deployment & Infrastructure as Code	9
5.10 Monorepo Architecture (pnpm Workspaces)	9
6. Phased Development Approach	10
7. Team Work Division	10
8. Complete Request-Response Flow	11
9. Conclusion	11

1. ABSTRACT

This report presents SatyaCheck, an AI-powered fake news detection and verification web application developed as a final year project. The system leverages three state-of-the-art large language models — OpenAI GPT, Google Gemini, and Anthropic Claude — to provide multi-model consensus-based fact verification. The application accepts news content in the form of text, URLs, headlines, and images, analyzes the credibility of the content, and provides detailed explanations for its verdicts. This report documents the complete backend engineering process including API design, database architecture, AI integration, middleware implementation, and cloud deployment. The project aligns with United Nations Sustainable Development Goal 16: Peace, Justice and Strong Institutions, specifically targeting the global challenge of misinformation.

2. PROJECT OVERVIEW & NOVELTY FACTOR

SatyaCheck is a full-stack web application that enables users to verify the authenticity of news content using artificial intelligence. Unlike traditional single-model fact-checkers, our system employs three independent AI models to arrive at a consensus verdict, significantly improving reliability and reducing false positives/negatives.

Key Features

- /detect** — Submit text, URL, headline, or image for AI-powered analysis
- /dashboard** — View statistics and recent analysis history
- /chat** — Conversational AI chatbot for follow-up questions
- /credibility** — Check credibility score of news sources
- /quiz** — Interactive media literacy quiz
- /trending** — Trending misinformation topics
- /education** — Media literacy resources and learning material

Novelty Factors

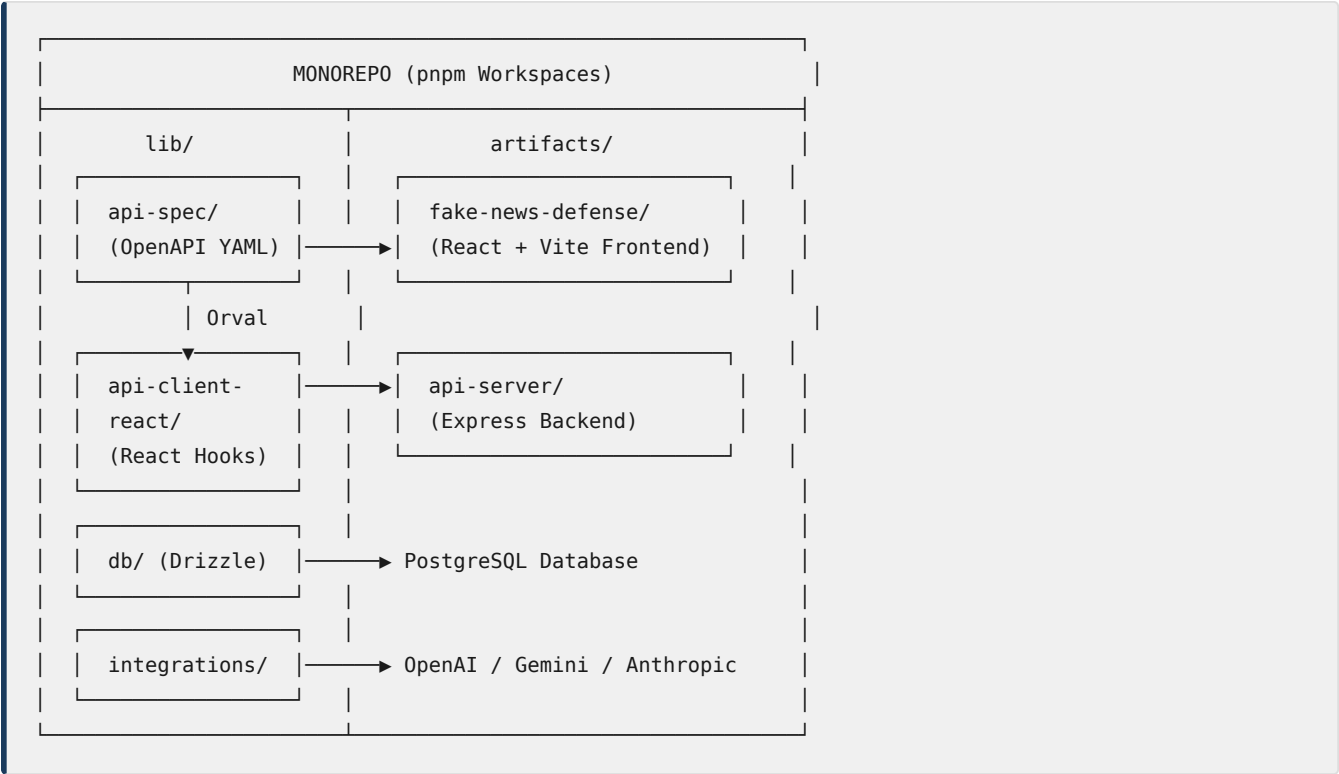
- Multi-Model AI Consensus:** Three AI models independently analyze and cross-verify content, reducing single-model bias
- Multi-Modal Input:** Accepts text, URL, headline, and image — not just plain text
- Explainable AI:** Per-claim fact breakdowns with individual confidence scores, not just a True/False verdict
- Interactive Follow-Up Chat:** Users can ask follow-up questions after analysis
- Education Layer:** Built-in media literacy module aligned with SDG goals

3. TECHNOLOGY STACK

Layer	Technology	Purpose
Frontend	React 18 + Vite (TypeScript)	User Interface
Backend	Express 5 (Node.js, TypeScript)	REST API Server
Database	PostgreSQL + Drizzle ORM	Persistent Data Storage
Primary AI	OpenAI GPT-5.2	Content Analysis & Reasoning
Verification AI	Google Gemini 2.5 Flash	Independent Fact Checking

Layer	Technology	Purpose
Chat AI	Anthropic Claude	Conversational Explanation
API Contract	OpenAPI 3.0 + Orval Codegen	Type-Safe API Generation
Validation	Zod	Runtime Schema Validation
Logging	Pino	Structured Production Logging
Monorepo	pnpm Workspaces	Multi-Package Management
Deployment	Render.com + render.yaml	Cloud Hosting & CI/CD

4. SYSTEM ARCHITECTURE



5. BACKEND ENGINEERING CONCEPTS LEARNED

5.1 REST API Design

Representational State Transfer (REST) is the architectural style used for designing our API. Each endpoint corresponds to a specific resource and uses HTTP methods to define the operation being performed.

GET	/api/stats	→ Retrieve analysis statistics
POST	/api/analyze	→ Submit content for analysis
GET	/api/history	→ Retrieve past analysis records
POST	/api/chat	→ Send a chat message to AI
GET	/api/trending	→ Fetch trending misinformation topics
POST	/api/credibility	→ Check source credibility

HTTP Method	Operation	Use Case in Project
GET	Read / Retrieve	Fetching history, stats, trending topics
POST	Create / Submit	Submitting content for analysis, chat messages
PUT	Update	Updating user preferences or records
DELETE	Remove	Deleting stored analysis records

Each endpoint follows standard HTTP status codes: 200 (Success), 400 (Bad Request), 404 (Not Found), and 500 (Internal Server Error).

5.2 OpenAPI Contract-First Development

We adopted a Contract-First development approach, where the API contract is defined before any code is written. The contract is specified in

```
lib/api-spec/openapi.yaml
```

using the OpenAPI 3.0 standard.

```
# Example from openapi.yaml
/api/analyze:
  post:
    summary: Analyze content for misinformation
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              content:
                type: string
              type:
                type: string
              enum: [text, url, headline, image]
    responses:
      '200':
        description: Analysis result
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AnalysisResponse'
```

The **Orval** code generation tool reads this YAML and automatically generates: (1) TypeScript React Query hooks for the frontend, and (2) Zod validation schemas for the backend. This eliminates the need to manually write and maintain duplicate type definitions across frontend and backend.

Key Benefit: Any change to the OpenAPI spec is propagated to both frontend and backend by re-running

```
pnpm --filter @workspace/api-spec run codegen
```

, ensuring the two layers are always in sync.

5.3 Input Validation using Zod

User-submitted data must never be trusted without validation. We used **Zod**, a TypeScript-first schema validation library, to validate all incoming request bodies before processing.

```
// Generated Zod schema from OpenAPI spec
const analyzeRequestSchema = z.object({
  content: z.string().min(1, "Content is required"),
  type: z.enum(["text", "url", "headline", "image"]),
});

// Usage in route handler
const parsed = analyzeRequestSchema.safeParse(req.body);
if (!parsed.success) {
  return res.status(400).json({ error: parsed.error.format() });
}
```

If a user submits an empty string or an unsupported content type (e.g., "video"), the server immediately returns a 400 error with a clear, descriptive message — without ever reaching the AI processing layer.

5.4 Database Design (PostgreSQL + Drizzle ORM)

We used **PostgreSQL** as our relational database and **Drizzle ORM** for type-safe database interactions. Drizzle allows writing database queries in TypeScript, which are then compiled to SQL.

```
// Schema definition in lib/db/src/schema/analyses.ts
export const analyses = pgTable("analyses", {
  id: serial("id").primaryKey(),
  content: text("content").notNull(),
  contentType: varchar("content_type", { length: 20 }).notNull(),
  verdict: varchar("verdict", { length: 20 }).notNull(),
  confidence: integer("confidence").notNull(),
  explanation: text("explanation"),
  createdAt: timestamp("created_at").defaultNow(),
});
```

Table	Purpose	Key Columns
analyses	Stores each analysis request and result	id, content, verdict, confidence, createdAt
conversations	Stores chat session metadata	id, analysisId, createdAt
messages	Stores individual chat messages	id, conversationId, role, content

5.5 Multi-Model AI Integration

The core innovation of SatyaCheck is its multi-model analysis pipeline. Three AI models process the same content independently and their results are combined to produce a high-confidence verdict.

```
// Simplified flow in artifacts/api-server/src/routes/analyze.ts

// Step 1: Primary analysis using OpenAI
const openaiResult = await openai.chat.completions.create({
  model: "gpt-5.2",
  messages: [{ role: "user", content: analysisPrompt }],
});

// Step 2: Cross-verification using Gemini
const geminiResult = await gemini.models.generateContent({
  model: "gemini-2.5-flash",
  contents: [{ role: "user", parts: [{ text: verifyPrompt }] }],
});

// Step 3: Combine results for consensus verdict
const finalVerdict = combineResults(openaiResult, geminiResult);
```

AI Model	Provider	Role in System
GPT-5.2	OpenAI	Primary analysis, reasoning, confidence scoring
Gemini 2.5 Flash	Google	Independent fact verification, claim-by-claim check
Claude	Anthropic	Conversational explanation in the chat interface

5.6 Middleware Architecture

Middleware functions are a central pattern in Express.js. Each incoming request passes through a chain of middleware before reaching the route handler. This keeps concerns separated and the code modular.

```
// Middleware chain in artifacts/api-server/src/app.ts
app.use(cors()); // 1. Cross-Origin Resource Sharing
app.use(pinoHttp({ logger })); // 2. HTTP request logging
app.use(express.json({ limit: "50mb" })); // 3. JSON body parsing
app.use("/api", router); // 4. Route handling
```

- **CORS:** Controls which domains are allowed to make requests to our API, preventing unauthorized cross-origin access
- **Pino HTTP:** Automatically logs every incoming request with method, URL, status code, and response time
- **Body Parser:** Parses the JSON request body and makes it available as a JavaScript object on

```
req.body
```

5.7 Structured Logging with Pino

Production applications require structured, queryable logs rather than simple

```
console.log
```

statements. We used **Pino**, a high-performance JSON logger for Node.js.

```
// Correct way – structured log in a route handler
req.log.info({
  event: "analysis_complete",
  verdict: result.verdict,
  confidence: result.confidence,
  durationMs: Date.now() - startTime,
});

// Wrong way – avoid in production
console.log("Analysis done"); // No structure, no queryability
```

Structured logs allow operations teams to filter, search, and monitor application behavior in production — for example, finding all requests where confidence was below 60%, or identifying slow AI responses.

5.8 Environment Variables & Secret Management

Sensitive credentials such as API keys and database connection strings must never be hardcoded in source code. We used environment variables, which are injected at runtime and excluded from version control.

```
# .env file – never committed to GitHub
AI_INTEGRATIONS_OPENAI_API_KEY=sk-...
AI_INTEGRATIONS_GEMINI_API_KEY=AIza...
AI_INTEGRATIONS_ANTHROPIC_API_KEY=sk-ant-...
DATABASE_URL=postgresql://user:pass@host/db
SESSION_SECRET=random-secure-string
```

```
// Accessed safely in code
const openai = new OpenAI({
  apiKey: process.env.AI_INTEGRATIONS_OPENAI_API_KEY,
  baseURL: process.env.AI_INTEGRATIONS_OPENAI_BASE_URL,
});
```

Security Note: Exposing API keys in source code can result in unauthorized usage, financial liability (billing charges), and data breaches. All secrets in this project are managed through environment variables and the Replit Secrets Manager.

5.9 Deployment & Infrastructure as Code

We configured the application for cloud deployment on Render.com using a

```
render.yaml
```

file — a declarative configuration that describes the entire infrastructure.

```
# render.yaml – Infrastructure as Code
services:
  - type: web
    name: satyacheck-api
    runtime: node
    buildCommand: pnpm install && pnpm --filter @workspace/api-server run build
    startCommand: node artifacts/api-server/dist/index.js
    envVars:
      - key: NODE_ENV
        value: production
      - key: DATABASE_URL
        fromDatabase:
          name: satyacheck-db
          property: connectionString

databases:
  - name: satyacheck-db
    plan: free
```

This approach, known as **Infrastructure as Code (IaC)**, ensures that the deployment environment is reproducible, version-controlled, and does not require manual configuration through a web dashboard.

5.10 Monorepo Architecture with pnpm Workspaces

Instead of maintaining separate repositories for the frontend and backend, we used a **monorepo** — a single repository containing multiple packages. This is the approach used by companies like Google, Meta, and Microsoft.

```
workspace/
├── lib/                # Shared packages (used by both frontend & backend)
│   ├── api-spec/      # Single source of truth – OpenAPI specification
│   ├── api-client-react/ # Auto-generated React Query hooks
│   ├── api-zod/        # Auto-generated Zod validation schemas
│   ├── db/             # Drizzle ORM schema and DB connection
│   └── integrations-*/ # AI client wrappers
├── artifacts/          # Deployable applications
│   ├── fake-news-defense/ # React + Vite frontend
│   └── api-server/       # Express.js backend
```

Key advantages: (1) Shared types prevent drift between frontend and backend, (2) A single

```
pnpm install
```

installs all dependencies, (3) Changes to shared libraries are immediately reflected in all consuming packages.

6. PHASED DEVELOPMENT APPROACH

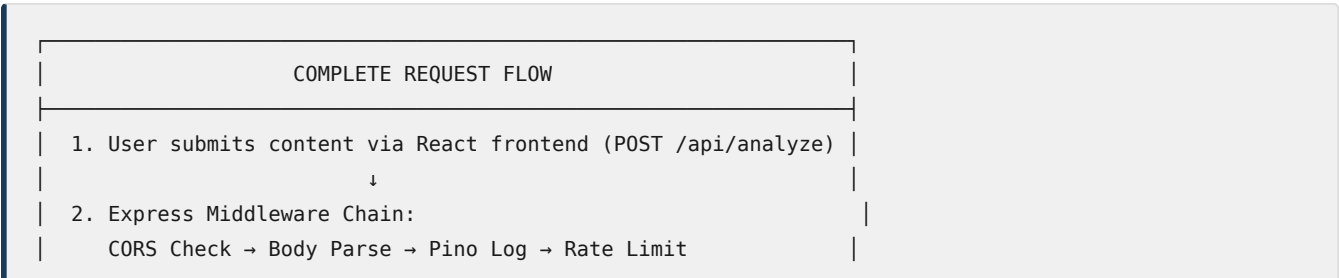
Our development followed a structured, phase-based approach where each phase unblocked the next, and independent tasks were executed in parallel to maximize efficiency.

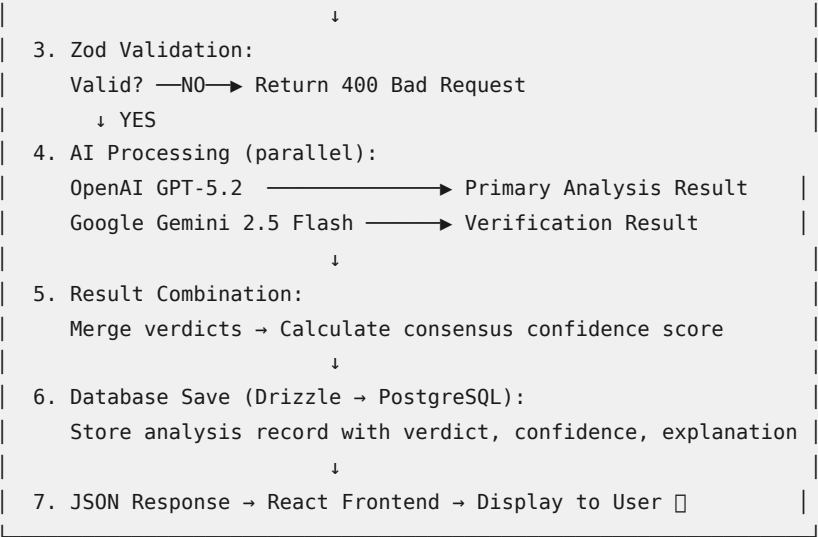
Phase	Activity	Why This Phase Came First
Phase 1	OpenAPI Specification	Blueprint for entire system — gates all subsequent development
Phase 2	Code Generation (Orval)	Produces typed hooks and schemas used by both frontend and backend
Phase 3	Frontend Build (React)	Can now use generated hooks; runs in parallel with backend
Phase 4	Backend Routes (Express)	Runs in parallel with frontend; frontend does not block backend
Phase 5	AI Integration	Routes exist first; AI logic slots cleanly into handlers
Phase 6	Database Schema & Migration	Routes call DB; schema finalized after route contracts are clear
Phase 7	Testing	All components integrated and verifiable end-to-end
Phase 8	Deployment	Stable, tested codebase deployed to production

7. TEAM WORK DIVISION

Phase	Student 1 (Backend)	Student 2 (Frontend)	Student 3 (AI/ML)	Student 4 (DB + DevOps)
Planning	Define API endpoints	Design page flow & UI wireframes	Research AI models	Plan database schema
OpenAPI Spec	□ Write all endpoint definitions	Review response shapes	Define AI response format	Suggest DB-aligned fields
Code Generation	Run codegen command	Verify generated hooks	Verify Zod schemas	—
Frontend	Wire API hooks in App.tsx	□ Build all pages & components	Design AI result UI	—
Backend Routes	□ Implement all /api routes	Test routes from frontend	Add AI calls in routes	Add DB calls to routes
AI Integration	Maintain route structure	Build loading/error states	□ Integrate OpenAI + Gemini + Claude	Save AI results to DB
Database	Use DB in route handlers	Build history & dashboard	Format AI output for DB	□ Write Drizzle schema & run migrations
Testing	Test API via Postman	Manual UI testing	Verify AI accuracy	Check DB queries & indexes
Deployment	Configure render.yaml	Optimize Vite build	Set production AI keys	□ Set up PostgreSQL on Render
Documentation	Write API documentation	Capture screenshots	Document AI pipeline	□ Write README & architecture diagram

8. COMPLETE REQUEST-RESPONSE FLOW





9. CONCLUSION

This project gave our team hands-on experience with professional, production-grade backend engineering practices. We learned not just how to write code, but how to architect a complete system — from API design and database modeling to AI integration and cloud deployment.

The key takeaways from this project are:

- A **Contract-First approach** (OpenAPI) dramatically reduces integration bugs between frontend and backend
- **Code generation** from a single source of truth eliminates duplication and keeps types consistent
- **Input validation** (Zod) is not optional — it is a fundamental security and data quality requirement
- **Multi-model AI consensus** produces more reliable results than relying on a single model
- **Monorepo architecture** with shared libraries enables a large codebase to remain maintainable and consistent
- **Infrastructure as Code** makes deployments reproducible and eliminates manual configuration errors

SatyaCheck demonstrates that modern web applications are not built tool by tool, but are architected as systems — where each component has a clear responsibility and interfaces cleanly with every other component. This is the essence of professional software engineering.

Student 1 (Backend Dev)

Signature: _____

Student 2 (Frontend Dev)

Signature: _____

Student 3 (AI/ML Engineer)

Signature: _____

Student 4 (DB + DevOps)

Signature: _____